An Optimized Adaptive Heun Integrator for Differentiable Simulation of Stiff Power Electronic Systems

Hamad Alduaij
School of Electrical, Computer and Energy Engineering
Arizona State University, Tempe, AZ
halduaij@asu.edu

Abstract

Differentiable ODE solvers are central to gradient-based training of neural controllers for power electronic systems. We identify a first-same-as-last (FSAL) implementation error in the widely used `torchdiffeq` adaptive Heun solver that evaluates the first stage at an incorrect point, inflating the error constant by 2–3×. We provide a corrected and optimized implementation with JIT-compiled step logic, branchless adaptive step-size control, gradient separation, and Horner-form dense output. On an 18-state system with stiff phase-locked-loop dynamics, the optimized solver achieves a 2.1× wall-clock speedup over the original `torchdiffeq` implementation while producing 2–3× smaller errors at the same step size. The code is open-source at `https://github.com/halduaij/inverter-control`.

1 Introduction

Gradient-based optimization of neural controllers for power systems requires differentiable time-domain simulation. The controller parameters appear inside the ODE right-hand side $f(t, y; \theta)$, and training proceeds by backpropagating a trajectory-level loss through the ODE integrator to obtain $\partial\mathcal{L}/\partial\theta$. Libraries such as `torchdiffeq` [1] provide adaptive-step ODE solvers within the PyTorch autograd framework and have become standard infrastructure for neural ODE training.

Power electronic systems, however, present a specific numerical challenge: the coexistence of fast inner control loops (phase-locked loops with bandwidth ∼900 rad/s, current controllers at ∼1 kHz) and slow electromechanical dynamics (governor response at ∼0.1–1 Hz). This moderate stiffness makes explicit methods viable—unlike the deeply stiff chemical kinetics problems that mandate implicit solvers—but demands that the explicit method be both correct and efficient. The generic Runge–Kutta infrastructure in `torchdiffeq` introduces per-step overhead (Butcher tableau iteration, Python-level branching for step control, redundant tensor allocation) that compounds over the thousands of steps needed for moderately stiff simulations.

In this note we report two findings regarding `torchdiffeq`'s adaptive Heun solver. First, we identify a FSAL (first-same-as-last) implementation error that evaluates k_1 at the wrong point,

inflating the per-step error constant by a factor of 2–3× while preserving second-order convergence. Second, and more significant in practice, we provide an optimized standalone implementation that eliminates the generic RK overhead through JIT-compiled step arithmetic, branchless adaptive control, and gradient separation.

The contributions are:

1. Identification and correction of the FSAL evaluation-point error in the `torchdiffeq` Heun solver (Section 3).
2. A JIT-compiled adaptive Heun solver with branchless step-size control, gradient separation, and Horner-form dense output that achieves 2.1× wall-clock speedup over `torchdiffeq` (Section 4).
3. Validation on an 18-state system with stiff PLL dynamics (Section 6).

2 Background: Adaptive Heun Method

Heun's method is the simplest second-order explicit Runge–Kutta scheme. Given an ODE $\dot{y} = f(t, y)$, one step from (t_n, y_n) with step size h proceeds as:

$$k_1 = f(t_n,\ y_n), \tag{1}$$

$$k_2 = f(t_n + h,\ y_n + h\,k_1), \tag{2}$$

$$y_{n+1} = y_n + \tfrac{h}{2}(k_1 + k_2). \tag{3}$$

The Butcher tableau is

$$\begin{array}{c|cc} 0 & 0 & 0 \\ 1 & 1 & 0 \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

with the embedded first-order (Euler) method $\hat{y}_{n+1} = y_n + h\,k_1$ providing a local error estimate

$$e_{n+1} = y_{n+1} - \hat{y}_{n+1} = \tfrac{h}{2}(k_2 - k_1). \tag{4}$$

2.1 FSAL Property

For some Runge–Kutta methods (e.g., Dormand–Prince), the last stage evaluation k_s is computed at the accepted solution y_{n+1}, so it can be reused as k_1 for the next step. This "first same as last" (FSAL) property saves one function evaluation per accepted step.

For Heun's method, k_2 is evaluated at the Euler predictor $\tilde{y} = y_n + h\,k_1$, which differs from the accepted solution y_{n+1} by $O(h^2)$. Reusing k_2 as k_1 for the next step therefore evaluates the first stage at the wrong point, introducing an $O(h^2)$ per-step error in k_1. Since k_1 is multiplied by h in the update formula, this contributes an $O(h^3)$ local truncation error per step—the same order as Heun's inherent truncation error—so the global convergence order remains 2. However, the

additional error increases the leading error constant by a factor of 2–3×, producing less accurate solutions at the same step size.

3 The FSAL Bug in torchdiffeq

The `torchdiffeq` library implements adaptive Heun as a subclass of its generic adaptive-step Runge–Kutta solver. The generic solver assumes the FSAL property holds and caches the last stage evaluation for reuse. For methods where FSAL is valid (Dormand–Prince 4/5), this is correct. For Heun's method, however, the cached k_2 was evaluated at $\tilde{y} = y_n + h\,k_1$, not at y_{n+1}. Reusing it as k_1 for the next step means

$$k_1^{(n+1)} = f(t_{n+1},\ \tilde{y}_n) \neq f(t_{n+1},\ y_{n+1}), \tag{5}$$

where $\tilde{y}_n - y_{n+1} = \frac{h}{2}(k_1 - k_2) = O(h^2)$. This shifts the evaluation point of k_1 by a second-order amount at each step. While this does not degrade the convergence order (both versions remain second-order), it inflates the error constant, producing 2–3× larger errors at any given step size (Table 1). The corrupted k_1 also affects the error estimator (4), which uses k_1 in the difference $k_2 - k_1$, potentially reducing the reliability of adaptive step-size selection.

The fix is straightforward: evaluate $k_1 = f(t_{n+1}, y_{n+1})$ fresh at each step, at the cost of one additional function evaluation per step (2 evaluations total, matching the theoretical cost of a two-stage method).

4 Implementation

Beyond the FSAL correction, we provide a standalone adaptive Heun solver that replaces the generic Runge–Kutta infrastructure with specialized routines. The `torchdiffeq` generic solver iterates over the Butcher tableau, allocates intermediate k-tensors, and uses Python-level branching for step acceptance—all unnecessary overhead for a two-stage method. The following optimizations eliminate this overhead.

4.1 JIT-Compiled Step Logic

All arithmetic in the step function—Euler prediction, Heun correction, error estimation, and step-size update—is decorated with `@torch.jit.script`, eliminating Python interpreter overhead for the inner loop. The Heun step is computed directly as $y_1 = y_0 + \frac{h}{2}(k_1 + k_2)$ without iterating over a Butcher tableau. Optionally, the entire step can be wrapped with `torch.compile` (PyTorch 2.0+) for further kernel fusion.

Algorithm 1 Corrected Adaptive Heun Step

Require: State y_n, time t_n, step size h, tolerances atol, rtol

$k_1 \leftarrow f(t_n, y_n)$ {Fresh evaluation (no FSAL reuse)}
$\tilde{y} \leftarrow y_n + h\, k_1$ {Euler predictor}
$k_2 \leftarrow f(t_n + h,\ \tilde{y})$
with `no_grad()`:
  $y_1 \leftarrow y_n + \frac{h}{2}(k_1 + k_2)$
  $e \leftarrow \frac{h}{2}(k_2 - k_1)$
  $r \leftarrow \mathrm{RMS}(e\,/\,\mathrm{tol})$
  $h_{\mathrm{next}} \leftarrow h \cdot \mathrm{clamp}(0.9\, r^{-0.5},\ 0.2,\ 10)$
  accept $\leftarrow (r \le 1)$
end
$y_1^{\mathrm{grad}} \leftarrow y_n + \frac{h}{2}(k_1 + k_2)$ {With gradients}
return y_1^{grad}, accept, h_{next}

4.2 Branchless Step-Size Control

The step-size update uses `torch.where` throughout to avoid Python-level branching that would cause CPU–GPU synchronization:

$$h_{n+1} = h_n \cdot \mathrm{clamp}\big(\sigma \cdot r_n^{-1/p},\ d_{\min},\ d_{\max}\big), \tag{6}$$

where $r_n = \|e_{n+1}\|_{\mathrm{RMS}}/\mathrm{tol}$ is the normalized error ratio, $p = 2$ is the method order, $\sigma = 0.9$ is the safety factor, $d_{\min} = 0.2$, and $d_{\max} = 10$. The tolerance is mixed absolute–relative: $\mathrm{tol}_i = \mathrm{atol} + \mathrm{rtol} \cdot \max(|y_n^{(i)}|, |y_{n+1}^{(i)}|)$.

A step is accepted if $r_n \le 1$ or h_n is already at the minimum allowed step size.

4.3 Dense Output

For saving solutions at prescribed output times without restricting the adaptive step size, we fit a quartic Hermite interpolant through y_n, y_{n+1}, $y_{\mathrm{mid}} = y_n + (h/2)\, k_1$, k_1, and k_2, evaluated via Horner's rule with `torch.addcmul` for fused multiply-add:

$$y(\tau) = c_0 + \tau\big(c_1 + \tau(c_2 + \tau(c_3 + \tau\, c_4))\big), \quad \tau = \frac{t - t_n}{h} \in [0, 1]. \tag{7}$$

4.4 Gradient Separation

The accept/reject decision (6) is non-differentiable. We run the control logic inside `torch.no_grad()` for efficiency, then recompute the accepted solution $y_{n+1} = y_n + (h/2)(k_1 + k_2)$ outside the no-grad block so that gradients flow through the Heun formula to f and ultimately to the controller parameters θ. Algorithm 1 summarizes one adaptive step.

Table 1: Global error vs. step size: original (FSAL reuse) vs. corrected. Both methods converge at order 2 ($\sim$4$\times$ error reduction per halving of h); the FSAL bug inflates the error constant by 2.2–2.5$\times$.

h	Original (FSAL)	Corrected	Error ratio
0.400	4.7×10^{-3}	2.1×10^{-3}	2.2$\times$
0.200	1.2×10^{-3}	4.9×10^{-4}	2.4$\times$
0.100	2.9×10^{-4}	1.2×10^{-4}	2.4$\times$
0.050	7.1×10^{-5}	2.9×10^{-5}	2.5$\times$
0.025	1.8×10^{-5}	7.1×10^{-6}	2.5$\times$

5 Application: Multi-Converter Power System

The target application is a multi-converter power system with distributed adaptive voltage-oriented control (dVOC). Each converter has voltage and current PI control loops, phase estimation, and filter dynamics. We benchmark on an 18-state system that captures the essential stiffness structure: six fast PLL states with eigenvalues near $-30 \pm 780j$ rad/s, six medium-speed filter states, and six slow power-sharing states at 1–10 rad/s, producing a stiffness ratio of $\sim$100–1000.

This stiffness ratio is mild enough that explicit methods remain competitive with implicit solvers (which would require Jacobian factorization at each step), but severe enough that solver overhead matters—thousands of steps are needed for a 100 ms simulation at tight tolerances. The per-step overhead of generic Runge–Kutta infrastructure (tableau iteration, tensor allocation, Python branching) accumulates to a measurable fraction of total wall-clock time.

The solver is embedded in a differentiable training loop where neural network gain schedulers are optimized via backpropagation through the ODE trajectory. Both solution accuracy and computational efficiency directly affect training: larger error constants produce noisier gradients, while per-step overhead limits the number of training iterations per hour.

6 Numerical Results

6.1 Convergence Order and Error Constants

We verify the convergence order and error constants on three test problems with known solutions, integrated with fixed step sizes. Table 1 reports results for $\dot{y} = -0.5\,y$, $y(0) = 1$, integrated to $t = 4$ (exact solution $y(4) = e^{-2}$).

Both versions exhibit the expected $\sim$4$\times$ error reduction when h is halved, confirming second-order convergence. The error ratio (FSAL to corrected) is consistently 2.2–2.5$\times$ across all step sizes, indicating that the FSAL bug inflates the leading error constant without changing the convergence order. We observe similar ratios on nonlinear ($\dot{y} = -y^2$, ratio 2.9–3.0$\times$) and oscillatory ($\ddot{y} + y = 0$, ratio 2.5$\times$) test problems.

Table 2: Wall-clock time: `torchdiffeq` vs. optimized solver (18-state system, 100 ms integration, CPU).

Solver	**Time (ms)**	**Speedup**	**Steps**
`torchdiffeq` (rtol=10^{-4})	755	1.0×	∼4300
Optimized (rtol=10^{-4})	350	2.2×	4290
`torchdiffeq` (rtol=10^{-6})	7328	1.0×	∼43,600
Optimized (rtol=10^{-6})	3441	2.1×	43,639

6.2 Performance vs. torchdiffeq

Table 2 compares wall-clock time for the optimized solver against `torchdiffeq`'s adaptive Heun on the 18-state stiff system (100 ms integration, 50 output points, CPU). The optimized solver achieves a consistent 2.1× speedup despite performing 2 function evaluations per step (vs. 1 for the FSAL-reusing original). The speedup comes from eliminating generic RK overhead: specialized two-stage step logic replaces Butcher tableau iteration, JIT-compiled arithmetic removes Python interpreter overhead, and branchless step-size control avoids CPU–GPU synchronization points.

Both solvers take approximately the same number of steps for a given tolerance, confirming that the speedup is due to reduced per-step overhead rather than algorithmic step count differences.

7 Conclusion

We identified a FSAL implementation error in the `torchdiffeq` adaptive Heun solver that evaluates the first stage at the Euler predictor rather than the accepted solution, inflating the error constant by 2–3×. The corrected and optimized implementation, available at `https://github.com/halduaij/inverter-control`, combines the FSAL fix with JIT-compiled step logic, branchless adaptive control, and gradient separation. On an 18-state system with stiff PLL dynamics, the optimized solver achieves 2.1× wall-clock speedup over `torchdiffeq` while producing more accurate solutions, directly improving both simulation throughput and gradient quality for neural controller training.

References

[1] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural ordinary differential equations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[2] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[3] J. R. Dormand and P. J. Prince, "A family of embedded Runge–Kutta formulae," *J. Comput. Appl. Math.*, vol. 6, no. 1, pp. 19–26, 1980.

[4] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed. Berlin: Springer, 1993.